Universidad de San Carlos de Guatemala
Facultad de Ingeniería Escuela de Ciencias y Sistemas
Organización de Lenguajes y Compiladores 1
Catedrático: Ing. Mario Jose Bautista Fuentes
Tutor académico: Cristian Noé Axpuac Aspuac



Manual Tecnico
Proyecto

OBJ-C

Grupo 6

Nombre:	Carné:
Ottoniel Fabricio Vásquez Pineda	202307820
Arelis Esther Orozco Hernández	202300465
Jennifer Anahi Vielman Monzón	201807293
Douglas Alberto Ajú Mucía	202300559

Guatemala, 27 de Junio de 2025

Introducción

En el presente documento, se describe la estructura y los demás aspectos técnicos del proyecto OBJ-C, el cual tiene como propósito implementar un analizador léxico sintáctico y semántico para la obtención de información y la generación de reportes comprensibles y útiles al usuario mediante una arquitectura cliente-servidor.

Se presentará a detalle no solo la estructura interna del programa sino también el proceso previo al análisis incluyendo las expresiones regulares empleadas en la etapa del análisis léxico, así como también la gramática utilizada definida e implementada para el análisis sintáctico. Asimismo se describirá la lógica desarrollada para la construcción e implementación del árbol sintáctico.

OBJETIVOS

Generales

Familiarizar al lector con la lógica planteada para el funcionamiento del sistema realizado mediante el uso del lenguaje de programación Python así como unas cuantas explicaciones de HTML5 y CSS, para la parte del servidor y el cliente respectivamente, así como la estructura de datos para el AST.

Específicos

• Mostrar la estructura del programa realizado.

• Dar a conocer con detalle los métodos y variables empleadas, así como su funcionalidad dentro del programa.

• Mostrar la gramática creada e implementada para la correcta ejecución del programa.

Especificaciones Técnicas

Requisitos del Hardware:

• Mínimo 1GB de memoria RAM

• Procesador Intel Core 2 Duo 2 o superior

• Espacio en disco duro: Mínimo 5 GB

Requisitos del Software:

• Tener Python instalado.

• Tener el IDE VisualStudioCode.

• Tener sistema operativo como Windows 10 o MAC OS

• Tener instaladas o con acceso a las siguientes librerías y paquetes: Graphviz, PLY

, Flask.

Lógica del programa:

Archivo	Descripción
AND.py	En esta clase se implementa la lógica del operador lógico AND que es conjugación, en la cual devuelve un resultado de la operación AND entre los valores.
NOT.py	En esta clase se implementa la lógica del operador lógico NOT que es negación, en la cual devuelve una negación lógica del valor.
OR.py	En esta clase se implementa la lógica del operador lógico OR que es disyunción, en la cual devuelve un resultado de la operación OR entre los valores.
UNARIO.py	En esta clase se implementa la lógica del operador unitario, en la cual este al encontrar un valor “-” a la par de otro valor, este le devuelve un valor negado.
XOR.py	En esta clase se implementa la lógica del operador lógico XOR que es exclusión, en la cual este devuelve un resultado de la operación XOR entre los valores.
CAsignacion.py	La clase presente, es para asignar valores de diferentes tipos a variables ya existentes, osea esta clase cambiará el dato que tenga el tipo de la variable.
CFor.py	Esta clase implementa la lógica de un ciclo llamado for, en la cual esta clase gestionará este ciclo, su inicialización, la condición, la actualización y su cuerpo del ciclo, en la cual se va creando un nuevo ámbito para cada iteración que haga y maneja también lo que es el break y continue por si viene en el ciclo.

CIncremeto.py	Esta clase contiene 2 clases diferentes, incremento y decremento, en la cual serían como los operadores ++ y –, en la cual esta clase va a modificar los valores de cada variables, o también vectores, en la cual puede aumentar el valor o disminuir, siempre y cuando sean numéricos.
CPrintln.py	Esta clase tiene la implementación de imprimir en consola, en la cual este evalúa las expresiones y va a mostrar su valor, también imprime vectores completos.
DeclaracionVector.py	Esta clase maneja lo que es la creación y declaración de los vectores, haciendo verificaciones de dimensiones y también de valores iniciales, también evaluará que hay valores no declarados y los tendrá como por defecto.
AsignacionVector.py	Clase que gestiona las asignaciones de los valores específicos de un vector, este solo asignará siempre y cuando el valor que se le asigna sea del mismo tipo que se haya declarado.
AccesoVector.py	Permite acceder a elementos dentro de un vector. Valida que el vector exista y que los índices están dentro del rango- Devuelve el valor almacenado si no hay errores y gestiona los accesos seguros,.
ReporteVector.py	Clase que genera reportes de los vectores, linealiza vectores multidimensionales, los convierte en row-major y da el reporte del vector.
Vector.py	Clase representa vectores de una o múltiples dimensiones, osea almacena todo los datos del vector, que son como el tipo, el identificador, las dimensiones, valores y el scope y verificación de los

	vectores para manipularlo.

CAST.py	Clase para la representación y construcción del árbol sintáctico gráficamente, crea un nodo con texto y estilos, luego una arista para la conexión entre nodos para de último generar un archivo en formato SVG con el AST. Contiene una función externa **generarGrafica** utilizada en cada clase para construir y renderizar el AST desde cualquier nodo raíz del programa.
CInstruccion.py	Clase base de tipo abstracta para realizar las instrucciones del lenguaje OBJ-C.
CAritmeticas.py	Utilizada para evaluar operaciones aritméticas utilizando el método interpretar () entre dos operandos (suma, resta, división, …) contiene componentes como operación: para la verificación del tipo de operación utilizando la clase Operadores. Operando1 y operando2: representa las expresiones a evaluar.
Operadores	Clase para enumerar las operaciones aritméticas permitidas por el programa, utilizadas como constantes dentro de la clase Aritmeticas.
CNativo.py	Representa los valores literales del programa, usa el método interpretar () para evaluar y retornar el valor nativo Su comportamiento es por un identificador, donde accede a el valor desde la tabla de símbolos, si se tratara

	de un booleano, lo interpreta y normaliza.
CBreak.py	Utilizada para representar la interrupción de los ciclos, teniendo de validación el que solo se pueda ejecutar dentro de bucles o estructuras switch. Teniendo como comportamiento, que si se utiliza fuera de un ciclo, deberá retornar un error.
CDeclaracion.py	Clase para el manejo de la declaración de variables, con valores o sin valores iniciales. El flujo de la clase consiste en evaluar si existe la expresión, verificar compatibilidad de tipo, registrar la variable en la tabla de símbolos.
CLlamada.py	Ejecuta un procedimiento previamente declarado. Verificando si el procedimiento existe, evaluando, asignando los parámetros y previniendo recursividad.
CProcedimientos.py	Define un procedimiento declarado por el usuario para posteriormente ser llamado por la clase llamada. Registrando lo primero en el árbol global, ejecuta instrucciones con un nuevo entorno y captura y acumula errores de haberlos encontrado durante la interpretación
SentenciaIf.py	Clase utilizada para la representación de las estructuras if, else y else if. Usa un método interpretar para el flujo de las instrucciones del programa: creando un nuevo entorno, evaluando que las condiciones sean booleanas, guardando errores semánticos si llegaran aparecer, soporta recursión, para trabajar con varias instancias.

CArbol.py	Clase que contiene toda la ejecución del programa, trabaja con atributos como instrucciones, funciones (procedimientos definidos por el usuario), procedimiento actual utilizado para evitar la recursividad en las llamadas a procedimientos.
CSimbolo.py	Guarda la información referente a una variable, parámetro o entidad del lenguaje en la tabla de símbolos. Contiene métodos de acceso como get/set.
CTipo.py - **tipoDato**	Se utiliza para las declaraciones, validación de tipo y comparación de compatibilidad entre parámetros y expresiones. Se representa y encapsula el tipo de datos que manejan las instrucciones y expresiones.
TablaSimbolo.py	Se usa para administrar el almacenamiento, actualización y recuperación de variables y funciones del lenguaje, emulando el comportamiento de un stack de entornos
SingletonMeta()	Metaclase para implementar el patrón Singleton, para utilizar una sola instancia de cualquier clase que utilice tablaSimbolo. Controla la creación de instancias y almacena internamente una sola instancia por clase
CRelacionales.py	Se utiliza para realizar comparaciones entre dos expresiones , retornando valores booleanos. Ayuda a la ejecución de las condiciones dentro de las estructuras if, while ,etc.

CContinue.py	Utilizada para el comportamiento de la instrucción continue dentro de los ciclos del compilador.
CSwitch.py	Se utiliza para la implementación de la instrucción switch - case. Usa el método interpretar para evaluar la expresión principal , compara los case y ejecuta las instrucciones.
CWhile.py	Su método interpreta y evalúa la condición como booleana, ejecuta las instrucciones internas respetando break y continue, actualizando la condición en cada ciclo, el método graficar construye en el AST un nodo “while” con dos hijos: la condición y el bloque de cuerpo.
CDoWhile.py	A diferencia de while, siempre ejecuta al menos una iteración antes de comprobar la condición. En interpretar ejecuta el cuerpo, luego evalúa la condición; si no es booleana o hay errores semánticos, lo reporta. Para el AST, graficar crea un nodo “DoWhile” con los subnodos correspondientes al cuerpo y a la condición.
seno.py	define la función nativa seno(expr). En interpretar obtiene el valor de la expresión, verifica que se anumérico (int o float), llama a math.sin y retorna el resultado. En graficar añade al AST un nodo “Seno” que apunta al subárbol de la expresión.
coseno.py	parecido al seno, solo que se llama a math.cos para calcular el coseno de un radian, verifica tipo numérico y retorna el valor.

inverso.py	Implementa Inv(entero). Toma un entero, extrae su valor absoluto en cadena, lo invierte y devuelve al entero preservando el signo. Rechaza argumentos no enteros con un error semántico.
shuffle.py	Toma como parámetro el nombre de un vector multidimensional ya declarado, lo aplana en orden y reordena sus elementos, reconstruye la estructura anidada con las dimensiones invertidas y devuelve un nuevo objeto Vector.
Sort.py	Verifica que la expresión sea numérica. Ordena los valores del vector unidimensional de forma ascendente por defecto o descendente según lo indicado.

Gramática

Terminales={

{ ID, ENTERO, DECIMAL, CADENA, CARACTER, BOOLEANO, TRUE, FALSE,

INT, DOUBLE, STRING, BOOL, CHAR,

IF, ELSE, FOR, WHILE, DO, BREAK, CONTINUE, SWITCH, CASE, DEFAULT,

PRINTLN, PROC, EXEC, VECTOR, SENO, COSENO, INV, SHUFFLE, SORT,

+, -, *, /, **, %, =, ++, --, ==, !=, <, <=, >, >=, &&, ||, ^, !,

;, :, ,, (,), {, }, [,] **}**

No terminales = {

inicio, instrucciones, instruccion,

declaracion, declaracion_vector, asignacion_vector,

asignacion, incremento, decremento,

if, if_simple, if_else, if_elseif, if_elseif_else,

while, do_while, for, for_init, for_update,

switch, lista_casos, caso,

break, continue, println,

expresion, expresion_vector, contenido_vector,

lista_valores, lista_vectores, literal, funcion_nativa,

bloque, tipos, dimension, dimensiones,

procedimiento, parametros, llamada, param_llamada,

empt**y }**

Inicio

```
<instrucciones> ::= <instrucciones> <instruccion>
                  | <instruccion>
                  | ε

<instruccion> ::= <declaracion> ;
                | <declaracion_vector>
                | <asignacion_vector>
                | <asignacion> ;
                | <incremento> ;
                | <decremento> ;
                | <if>
                | <while>
                | <do_while>
                | <for>
                | <switch>
                | <break> ;
                | <continue> ;
                | <println> ;
                | <bloque>
                | <procedimiento>
                | <llamada> ;
                | ;
```

Instrucción

```
<instrucciones> ::= <instrucciones> <instruccion>
                  | <instruccion>
                  | ε

<instruccion> ::= <declaracion> ;
                | <declaracion_vector>
                | <asignacion_vector>
                | <asignacion> ;
                | <incremento> ;
                | <decremento> ;
                | <if>
                | <while>
                | <do_while>
                | <for>
                | <switch>
                | <break> ;
                | <continue> ;
```

| <println> ;
| <bloque>
| <procedimiento>
| <llamada> ;
| ;

Declaración

<declaracion> ::= <tipos> ID = <expresion>
| <tipos> ID

<tipos> ::= int | float | str | char | bool

Asignación

<asignacion> ::= ID = <expresion>

Incremento/Decremento

<incremento> ::= ID ++
| ID <dimension> ++

<decremento> ::= ID --
| ID <dimension> --

Vectores

<declaracion_vector> ::= vector [<tipos>] ID (<dimensiones>) =
<expresion_vector> ;
| vector [<tipos>] ID (<dimensiones>) ;

<asignacion_vector> ::= ID <dimension> = <expresion> ;

<dimension> ::= [ENTERO]
| <dimension> [ENTERO]

<dimensiones> ::= ENTERO
| <dimensiones> , ENTERO

<expresion_vector> ::= [<contenido_vector>]

<contenido_vector> ::= <lista_valores>
| <lista_vectores>

<lista_valores> ::= <expresion>
| <expresion> , <lista_valores>

<lista_vectores> ::= [<lista_valores>]
| [<lista_vectores>]
| [<lista_valores>] , <lista_vectores>
| [<lista_vectores>] , <lista_vectores>

Expresión

```
<expresion> ::= <expresion> + <expresion>
          | <expresion> - <expresion>
          | <expresion> * <expresion>
          | <expresion> / <expresion>
          | <expresion> ** <expresion>
          | <expresion> % <expresion>
          | - <expresion>
          | ( <expresion> )
          | <valor_primitivo>
          | ID
          | <expresion> || <expresion>
          | <expresion> && <expresion>
          | <expresion> ^ <expresion>
          | ! <expresion>
          | <expresion> < <expresion>
          | <expresion> <= <expresion>
          | <expresion> > <expresion>
          | <expresion> >= <expresion>
          | <expresion> == <expresion>
          | <expresion> != <expresion>
          | <funcion_nativa>
          | ID <dimension>
          | sort ( <expresion> )
          | sort ( <expresion> , <expresion> )
          | shuffle ( ID )

<valor_primitivo> ::= ENTERO | DECIMAL | CADENA | CARACTER | TRUE |
FALSE | BOOLEANO | CHAR
```

Función Nativa

```
<funcion_nativa> ::= seno ( <expresion> )
              | coseno ( <expresion> )
              | inv ( <expresion> )
              | shuffle ( ID )
              | sort ( <expresion> )
              | sort ( <expresion> , <expresion> )
```

IF

<if> ::= if (<expresion>) { <instrucciones> }
| if (<expresion>) { <instrucciones> } else { <instrucciones> }
| if (<expresion>) { <instrucciones> } else if (<expresion>) { <instrucciones> }
| if (<expresion>) { <instrucciones> } else if (<expresion>) { <instrucciones> } else { <instrucciones> }

While/Do While

<while> ::= while (<expresion>) <bloque>

<do_while> ::= do <bloque> while (<expresion>) ;

For

<for> ::= for (<for_init> ; <expresion> ; <for_update>) <bloque>

<for_init> ::= <declaracion>
| <asignacion>
| <incremento>
| <decremento>
| ε

<for_update> ::= <asignacion>
| <incremento>
| <decremento>
| ε

Switch

<switch> ::= switch (<expresion>) { <lista_casos> }

<lista_casos> ::= <lista_casos> <caso>
| <caso>

<caso> ::= case <expresion> : <instrucciones>
| default : <instrucciones>

Break/Continue

<break> ::= break
<continue> ::= continue

Println

<println> ::= println (<expresion>)

Bloques

<bloque> ::= { <instrucciones> }

<procedimiento> ::= proc ID (<parametros>) { <instrucciones> }

<parametros> ::= <tipos> : ID
| <tipos> : ID , <parametros>
| ε

Llamada

<llamada> ::= exec ID (<param_llamada>)

<param_llamada> ::= <literal>
| ID
| BOOLEANO
| <param_llamada> , <literal>
| <param_llamada> , ID
| <param_llamada> , BOOLEANO
| ε

<literal> ::= ENTERO | DECIMAL | CARACTER | CADENA | TRUE | FALSE

Conclusión

Este proyecto demuestra la implementación práctica de un lenguaje interpretado construido desde cero , combinando el análisis léxico, sintáctico y semántico mediante PLY juntando las instrucciones en un solo entorno unificado, permitiendo un control estructurado de la ejecución.

La implementación de estructuras como vectores, procedimientos y funciones nativas permite mejorar la funcionalidad. Junto al uso del AST, los diferentes reportes como la tabla de símbolos y detección de errores hace que sea más comprensible entender cómo operan los compiladores. Cumpliendo así la finalidad de este proyecto que busca ser una herramienta formativa valiosa para los estudiantes.